LESSON THREE

MERGE CONFLICT AND HOW TO RESOLVE IT

3.1 Meaning of Merge

I learned that merging is the process of combining changes from one branch into another branch. It is commonly used when a developer completes a feature and wants to add it to the main project.

For example, if I create a branch called login-feature and finish developing the login page, I can merge that branch into the main branch.

3.2 Meaning of Merge Conflict

I learned that a merge conflict happens when Git cannot automatically combine changes made to the same part of a file.

This usually happens when two or more developers edit the same lines of a file differently and try to merge their changes.

For example, imagine two developers working on the same website.

- Developer A changes the homepage heading to "Welcome to Our Website".
- Developer B changes the same heading to "Welcome to Our Company".

When both changes are merged, Git may not know which heading should be kept. This situation creates a merge conflict.

3.3 Causes of Merge Conflicts

I learned that merge conflicts can happen for several reasons:

1. Two developers edit the same line of a file.
2. Two developers make different changes to the same section.
3. A file is deleted by one developer but modified by another.
4. Changes are made in different branches and later merged.
5. Developers work with outdated versions of files.

3.4 How to Identify a Merge Conflict

When a merge conflict occurs, Git may display a message indicating that a conflict has been detected.

It may also insert special conflict markers into the affected file.

Example:

```
<<<<<<< HEAD
Welcome to Our Website
=======
```

```
Welcome to Our Company
>>>>>>> login-feature
```

The markers have the following meanings:

- `<<<<<<< HEAD` indicates the beginning of the current branch's changes.
- `=======` separates the two conflicting versions.
- `>>>>>>> login-feature` indicates the end of the incoming branch's changes.

3.5 Steps for Resolving a Merge Conflict

I learned that a merge conflict can be resolved manually by following these steps:

1. Open the file where the conflict occurred.
2. Read both versions of the conflicting changes.
3. Decide which version should be kept.
4. Edit the file to produce the correct final content.
5. Remove all conflict markers.
6. Save the file.
7. Add the corrected file to Git.
8. Create a commit to complete the resolution.

Example of the corrected file:

```
Welcome to Our Website
```

After resolving the conflict, I can use these commands:

```
git add .
git commit -m "Resolved merge conflict"
```

3.6 How to Prevent Merge Conflicts

I learned that merge conflicts cannot always be avoided, but they can be reduced by following good development practices.

These include:

1. Communicating with other team members.
2. Dividing project tasks properly.
3. Pulling the latest changes before starting work.
4. Avoiding unnecessary changes to other members' files.
5. Creating separate branches for different features.
6. Reviewing changes before merging them.

Conclusion

From this lesson, I understood that a merge conflict occurs when Git cannot automatically combine different changes. I also learned that resolving conflicts requires checking the affected files, choosing the correct changes and saving the corrected version.